

多言語対応（i18n）実装ガイド

本プロジェクトにおける英語・中国語・日本語の翻訳実装をまとめたドキュメントです。

使用ライブラリ（4点セット）

ライブラリ	役割
i18next	コアライブラリ
react-i18next	React用フック（ <code>useTranslation</code> , <code>Trans</code> ）
i18next-browser-languagedetector	ブラウザ言語の自動検出
i18next-http-backend	JSONファイルの動的読み込み

```
npm install i18next react-i18next i18next-browser-languagedetector i18next-http-backend
```

ファイル構成

```
public/
├── locales/
│   ├── en.json  ← 英語テキスト一覧
│   ├── ja.json  ← 日本語テキスト一覧
│   └── zh.json  ← 中国語テキスト一覧
└── src/
    ├── i18n.js      ← 初期化設定
    ├── App.jsx       ← Suspense（ローディング画面）を設定
    └── components/
        ├── Header.jsx ← 言語切り替えUI（ドロップダウン）
        └── 各コンポーネント ← useTranslation()で翻訳を呼び出す
```

翻訳JSONの構造

ネストされたキー構造でテキストを管理します。

```
// en.json
{
  "header": {
    "card": "Okami Card"
  },
}
```

```
"homePage": {
  "section1": {
    "title": "Welcome",
    "features": [
      { "title": "Feature A", "description": "..."}
    ]
  }
}
```

- キーは ページ名.セクション名.要素名 の階層で統一する
- 複数行テキストは \n で改行指定
- 配列も扱える（後述の `returnObjects` オプションで取得）

初期化 (`src/i18n.js`)

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';
import HttpBackend from 'i18next-http-backend';

i18n
  .use(HttpBackend)
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    fallbackLng: 'ja', // 翻訳がなければ日本語にフォールバック
    debug: false,
    interpolation: {
      escapeValue: false // ReactはデフォルトでXSSを防ぐため不要
    },
    detection: {
      order: ['localStorage', 'navigator'], // 言語の検出順
      caches: ['localStorage'] // 選択した言語をlocalStorageに保存
    },
    backend: {
      loadPath: '/locales/{{lng}}.json' // JSONの読み込みパス
    },
    react: {
      useSuspense: true
    }
  });

export default i18n;
```

エントリーポイントでimportします。

```
// src/main.jsx
import './i18n';
```

App.jsx での Suspense 設定

JSONの非同期ロード中にローディング画面を表示するため、`<Suspense>` でラップします。

```
import { Suspense } from 'react';

const LoadingFallback = () => (
  <div className="min-h-screen flex items-center justify-center">
    <div className="w-12 h-12 border-4 border-cyan-500 border-t-transparent rounded-full animate-spin" />
  </div>
);

function App() {
  return (
    <Suspense fallback={<LoadingFallback />}>
      <Router>
        {/* ルート定義 */}
      </Router>
    </Suspense>
  );
}
```

コンポーネントでの翻訳パターン（4種類）

① 基本パターン（最多）

```
import { useTranslation } from 'react-i18next';

const MyComponent = () => {
  const { t } = useTranslation();
  return <h1>{t('header.card')}</h1>;
};
```

② 配列・オブジェクトを返すパターン

JSONに配列が入っている場合は `returnObjects: true` で取得します。

```
const { t } = useTranslation();
const features = t('homePage.section1.features', { returnObjects: true });
```

```
return features.map((feature, index) => (  
  <div key={index}>  
    <h3>{feature.title}</h3>  
    <p>{feature.description}</p>  
  </div>  
));
```

③ HTMLタグを含むパターン (Trans コンポーネント)

テキスト中にスタイル付きタグを挿入したい場合に使います。

```
// en.json  
{  
  "description": "To use the card, <accent>OKM is required</accent>."  
}
```

```
import { Trans } from 'react-i18next';  
  
<Trans  
  i18nKey="homePage.description"  
  components={{  
    accent: <span className="text-red-600 font-bold" />  
  }}  
</>
```

④ 言語によって画像・処理を切り替えるパターン

```
const { i18n } = useTranslation();  
  
const imageMap = {  
  ja: 'imgs/card-jp.png',  
  en: 'imgs/card-en.png',  
  zh: 'imgs/card-zh.png',  
};  
  
<img src={imageMap[i18n.language] ?? imageMap.ja} />
```

言語切り替えUI (Header.jsx)

`i18n.changeLanguage(lang)` を呼ぶだけで全コンポーネントに反映されます。

```
import { useTranslation } from 'react-i18next';

const Header = () => {
  const { i18n } = useTranslation();

  const changeLanguage = (lang) => {
    i18n.changeLanguage(lang);
  };

  return (
    <div>
      {[ 'ja', 'en', 'zh' ].map(lang => (
        <button
          key={lang}
          onClick={() => changeLanguage(lang)}
          style={{ fontWeight: i18n.language === lang ? 'bold' : 'normal' }}
        >
          {lang}
        </button>
      ))}
    </div>
  );
};
```

別プロジェクトへの導入ステップ

1. ライブラリをインストール

```
npm install i18next react-i18next i18next-browser-languagedetector i18next-http-backend
```

2. **public/locales/** に翻訳JSONを作成 (**en.json**, **ja.json**, **zh.json**)
3. **src/i18n.js** を作成し、**main.jsx** でimport
4. **App.jsx** を **<Suspense>** でラップ (非同期ロード対応)
5. 各コンポーネントで **useTranslation()** を使用してテキストを翻訳
6. 言語切り替えボタンを実装 (**i18n.changeLanguage(lang)** を呼ぶだけ)

設計上の選択肢 (案出し参考)

観点	本実装の選択	代替案
翻訳データの場所	public/locales/*.json (実行時に動的ロード)	src/ 内にimportして静的バンドル

観点	本実装の選択	代替案
言語設定の保持	localStorage	Cookie、URLパラメータ (/en/about)
言語別コンテンツ	JSON翻訳 + JSオブジェクトで言語分岐	完全にJSONで管理
言語別画像	ファイル名を言語ごとに用意してマッピング	共通画像を使い回す
フォールバック 言語	日本語 (ja)	英語 (en) が一般的